ANTLR

استاد درس : دکتر سوادی
دانشگاه فردوسی مشهد

مسئول پروژه : امیررضا خادم پیر
مهلت تحویل : 1404/00/00

شرح پروژه :

در این پروژه هدف پیاده سازی بخشی از ساختار کلی یک زبان ساده برنامه نویسی و رسم درخت کدهای نوشته شده به این زبان است.

ساختار زبان :

1. هر برنامه می تواند شامل یک کلاس و یا چند تابع و دستوراتی برای اضافه کردن کتابخانه ها باشد.
   دستورات اضافه کردن کتابخانه پیش از سایر دستورات می آیند و در آخر نام هر کتابخانه kh. وجود دارد.
2. هرکلاس می تواند تعریف آرایه، شامل توابع و تعریف متغییر باشد.
3. کامنت گذاری به دو صورت single-line و multi-line تعریف می شود و خطوط کامنت نباید در درخت ترسیم شوند.
   * کاراکتر مشخص کننده کامنت sinlge-line باید با علامت "#" مشخص شود.
   *کاراکتر مشخص کننده شروع کامنت multi-line باید با علامت "#$" و مشخص کننده پایان کامنت با علامت "$#" مشخص شود.
   مثال:

```
#$ Don’t use ChatGPT
for your project #$
```

4. کلمات کلیدی case-senstive نیستند.
5. قوانین نامگذاری :
   I. با رقم شروع نمی شوند.
   II. متشکل از حروف کوچک و بزرگ لاتین، ارقام و کاراکتر"@" و "%" می باشند.
   III. کلمات مشخص شده با رنگ سبز در این فایل، کلمات کلیدی هستند و نمی توانند نام متغییر باشند. بخش هایی که با رنگ بنفش مشخص شده اند وجود آنها اختیاری است

(یعنی باید به گونه ای برنامه را بنویسید که با وارد کردن یا نکردن آن بخش در تست کیس مشکلی پیش نیاید).

IV. حروف اول و دوم **نام کلاس** باید Capital باشد.

6. برای پایان دستورات باید از علامت ":!;" استفاده کنید.
7. به جای "[]" و "()" می بایست از "!!" استفاده کنید.
8. در تعریف متغییر می بایست نوع متغییر با متغییر وارد شده یکسان باشد.
   به عنوان مثال "kh" = Int sum قابل قبول **نیست**.

دستورات اضافه کردن کتابخانه ها:

import ! <modul_name> ! ;!:

From ! <modul_name> ! import <function_name> ;!:

Example:

Include ! math.kh ! ;!:

From ! math.kh ! Include delta ;!:

تعریف توابع:

✓ درصورت وجود var_type2 ، باید با var_type1 یکسان باشد.

<fun_type> Fun <function_name> <var_type1> ! <parameter_name>, <parameter_name> ! :{

<statements> ;!:

RETURN <var_type2> <parameter_name> ;!:

}:

Example1:

Public Fun text string ! text1 ! :{

Text1 = "hello dude" ;!:

```
RETURN ;!: }:
```

Exmple2:

```
Fun sum int !num1, num2! :{
Number3 = num1 + num2 ;!:
RETURN int Number3;!:
}:
```

تعريف متغيير:

```
<var_type> <var_name> = <value> ;!:
```

Example:

```
Int sum = 0 ;!:
String text = “hello word”
```

دستورات حلقه:

```
While ! <condition> ! :{
<statements> ;!:
}:
```

```
For <name> in <iterator_name>:{
<statements> ;!:
}:
```

For <name> in range !<value>,<var_name>! :{

<statements> ;!:

}:

Example:

While !true! :{

Print!i! ;!:

i = i+1 ;!:

}:

For x in arr:{

Print!x!

}:

For x in Range !6!:{

Print!x!

}:

تعریف آرایه :

- arr_type1 و arr_type2 باید یکسان باشند.

<arr_type1> <arr_name> = ! <size> ! <arr_type2> {<value> , <value>}

Example:

int arr1 = !3! int {0, 1, 2}

char arr2 = !4! char

~~char arr2 = !4! int~~

~~int arr1 = !a! int~~ (size just can be Integer)

دستورات شرط : (else if و else کلمات کلیدی هستند)

```
If ! <condition> ! :{
        <statements> ;!:
 }:
else if ! <condition> ! :{
        <statements> ;!:
}:
else :{
        <statements> ;!:
}:
```

Example:

```
If ! s == 5 ! :{
s--;!:
else if ! s < 5 ! :{
s++;!:
}:
```

Switch case:

```
sw !<var_name>! :{
        case <value> :
                <statements> ;!:
                break ;!:
         default:!
                <statements> ;!:
                break ;!:
                !
}:
```

Example:

```
Sw ! count ! :{
        case 0:
                RETURN 0 ;!:
        case 1:
                RETURN 1 ;!:
        dEfault:!
                RETURN “pain” ;!:
                !
```

کلاس ها:

```
CLass <var_name> :{
        <statements> ;!:
}:
```

Example:

```
ClasS ANimal :{
Int t = 5 ;!:
Fun cat string ! text1 ! :{
catName = text1 ;!:
RETURN;!:
}:}:
```

کار با فایل :

(method می تواند r به معنای خواندن و یا w به معنای نوشتن باشد.)

```
<var_name> = open!"<var_name>.txt", "<method>"! ;!:
<var_name>.write("<text>") ;!:
<var_name>.close() ;!:
```

Example:

```
file = open("example.txt","w") ;!:
 file.write("example@gmail") ;!:
 file.close() ;!:
```

exceptions:

```
try:{
        <statements> ;!: }:
except <var_name>:{
        <statements> ;!: }:
```

Example:

```
try:{
        t = t + 1 ;!:
}:
except valError :{
print!"ERROR"! ;!: }:
```

انواع عملگرهای زبان :

انواع عملگرهای مورد استفاده در زبان که باید تعریف شده و به ترتیب ذکر شده زیر شناسایی و مورد استفاده قرار گیرند، عبارتند از:

- **
- * / %
- - + ++ --
- or and
- == !=
- <= =>
- < >

توضیحات تکمیلی:

- تمیزی کد شما شامل نمره پروژه خواهد بود.
- در کنار گرامر خود، حتما یک یا چند نمونه برنامه زبانی که برای آن گرامر نوشته اید قرار دهید تا قابلیت های مختلف گرامرتان را نمایش دهد. درصورت عدم وجود تست کیس ، نمره کسر خواهد شد.
- فقط فایل گرامر(.g) و نمونه برنامه های خود (.txt) را در قالب یک فایل zip با نام Antlr_lastname_studentID در سامانه vu آپلود کنید.
- تحویل پروژه بعد از مهلت مشخص شده نمره ای نخواهد داشت.
- در هنگام تحویل پروژه، لازم است به گرامر خود تسلط کافی داشته باشید و در صورت نیاز، بتوانید تغییراتی در آن ایجاد کنید.

- همچنین لازم به ذکر است پروژه باید به صورت انفرادی انجام شود و در صورت دیده شدن هرگونه تخلف (استفاده از ChatGPT و...) و تقلب، برای فرد یا افراد مربوطه **نمره -100 ثبت خواهد شد**.
- اگر هرگونه سوال و یا ابهامی داشتید می توانید به آیدی @amirr_khm تلگرام بنده پیام بدید.

موفق باشید .